

[VN] Làm sao để xử lý distributed transaction? So sánh 2PC vs Saga pattern.

[EN] How do you handle distributed transactions? Compare 2PC vs Saga pattern.

[VN] Để xử lý distributed transaction trong hệ thống microservices, tôi thường tránh transaction phân tán theo kiểu truyền thống và ưu tiên thiết kế theo eventual consistency. Có hai cách phổ biến là 2PC và Saga pattern. 2PC (Two-Phase Commit) hoạt động theo hai bước: prepare và commit, có một coordinator đảm bảo tất cả service cùng commit hoặc cùng rollback. Cách này đảm bảo strong consistency nhưng nhược điểm là chậm, dễ bị block nếu một service bị lỗi và không phù hợp với hệ thống lớn, scale cao. Saga pattern chia transaction lớn thành nhiều transaction nhỏ, mỗi service tự commit dữ liệu của mình và nếu có lỗi thì thực hiện compensating action để rollback logic. Saga có thể triển khai theo choreography (event-driven) hoặc orchestration (có một service điều phối). Saga linh hoạt, scale tốt và phù hợp microservices, nhưng chỉ đảm bảo eventual consistency và cần thiết kế logic bù trừ cẩn thận.

[EN] To handle distributed transactions in microservices, I usually avoid traditional distributed locking and prefer eventual consistency design. Two common approaches are 2PC and Saga pattern. 2PC (Two-Phase Commit) works in two steps: prepare and commit, with a coordinator ensuring all services either commit or rollback together. It provides strong consistency but is slow, can block if one service fails, and is not suitable for large-scale systems. Saga pattern breaks a large transaction into multiple local transactions; each service commits its own data and, if an error occurs, executes a compensating action to logically rollback. Saga can be implemented using choreography (event-driven) or orchestration (central coordinator). Saga is more flexible and scalable for microservices but only guarantees eventual consistency and requires careful design of compensation logic.

[VN] Circuit Breaker là gì? Khi nào cần?

[EN] What is a Circuit Breaker? When is it needed?

[VN] Circuit Breaker là một pattern giúp bảo vệ hệ thống khi gọi service bên ngoài. Nó hoạt động giống như cầu dao điện: khi số lượng lỗi hoặc timeout vượt ngưỡng cho phép, circuit sẽ mở (open) và tạm thời chặn các request mới để tránh làm quá tải service đang lỗi. Sau một khoảng thời gian, nó chuyển sang trạng thái half-open để thử lại một số request; nếu thành công thì đóng lại (closed), nếu vẫn lỗi thì tiếp tục mở. Circuit Breaker giúp giảm cascading failure và tăng độ ổn định của hệ thống. Ta cần dùng khi hệ thống có nhiều service gọi lẫn nhau qua network, đặc biệt trong microservices hoặc khi gọi API bên thứ ba, nơi có nguy cơ timeout hoặc downtime cao.

[EN] Circuit Breaker is a pattern that protects the system when calling external services. It works like an electrical switch: when the number of failures or timeouts exceeds a

threshold, the circuit opens and temporarily blocks new requests to prevent overloading the failing service. After a period of time, it moves to half-open state to test a few requests; if they succeed, it closes again, if not, it stays open. Circuit Breaker helps prevent cascading failures and improves system stability. We need it when services communicate over the network, especially in microservices architecture or when calling third-party APIs where timeouts and downtime may happen.

[VN] Idempotency trong API là gì? Áp dụng vào payment/booking như thế nào?

[EN] What is idempotency in APIs? How is it applied to payment/booking?

[VN] Idempotency trong API có nghĩa là khi một request được gửi nhiều lần với cùng một dữ liệu, kết quả cuối cùng vẫn chỉ tạo ra một tác động duy nhất trên hệ thống. Nói cách khác, gọi lại nhiều lần không được tạo ra dữ liệu trùng lặp. Trong payment hoặc booking, điều này rất quan trọng vì nếu client retry do timeout hoặc mất mạng, hệ thống không được trừ tiền hai lần hoặc tạo hai booking. Thực tế, tôi sẽ dùng idempotency key do client gửi lên (ví dụ một unique request ID) và lưu lại trong database cùng với trạng thái xử lý. Nếu hệ thống nhận lại request với cùng idempotency key, nó sẽ trả về kết quả cũ thay vì xử lý lại. Cách này giúp tránh double charge, duplicate booking và đảm bảo an toàn dữ liệu.

[EN] Idempotency in API means that when the same request is sent multiple times with the same data, the final result only creates one effect in the system. In other words, repeated calls should not create duplicate data. In payment or booking systems, this is very important because if the client retries due to timeout or network issues, the system must not charge twice or create two bookings. In practice, I use an idempotency key provided by the client (for example a unique request ID) and store it in the database with the processing status. If the system receives another request with the same idempotency key, it returns the previous result instead of processing it again. This prevents double charge, duplicate booking, and ensures data safety.

[VN] Nếu một service downstream bị chậm, hệ thống sẽ bị ảnh hưởng ra sao? Cách mitigate?

[EN] If a downstream service is slow, how will the system be affected? How can it be mitigated?

[VN] Nếu một service downstream bị chậm, các service gọi đến nó sẽ phải chờ lâu hơn, dẫn đến tăng latency toàn hệ thống. Điều này có thể gây timeout, chiếm dụng thread hoặc connection pool, và tạo ra cascading failure nếu nhiều request bị treo cùng lúc. Trong trường hợp xấu, toàn bộ hệ thống có thể bị quá tải hoặc không phản hồi. Để mitigate, tôi áp dụng timeout hợp lý để không chờ vô hạn, dùng circuit breaker để ngắt tạm thời khi lỗi vượt ngưỡng, và retry có kiểm soát với backoff. Ngoài ra, có thể dùng asynchronous

communication để giảm phụ thuộc đồng bộ, cache dữ liệu nếu phù hợp, và giới hạn số lượng request bằng rate limiting hoặc bulkhead pattern để cô lập lỗi.

[EN] If a downstream service becomes slow, the calling services will experience higher latency, which can increase response time for the whole system. This may cause timeouts, exhaust threads or connection pools, and lead to cascading failures if many requests are blocked at the same time. In the worst case, the entire system can become overloaded or unresponsive. To mitigate this, I configure proper timeouts to avoid waiting indefinitely, use circuit breaker to stop calls when failures exceed a threshold, and apply controlled retries with backoff. In addition, I may use asynchronous communication to reduce synchronous dependency, cache data when possible, and limit requests using rate limiting or the bulkhead pattern to isolate failures.

[VN] Idempotency Key vs Trace ID

[EN] Idempotency Key vs Trace ID

[VN] Idempotency Key là một key duy nhất được client gửi kèm request để đảm bảo một operation chỉ được thực hiện một lần, ngay cả khi request bị gửi lại do retry hoặc lỗi mạng. Server sẽ lưu key này cùng kết quả xử lý; nếu nhận lại cùng key, server sẽ trả về kết quả cũ thay vì thực hiện lại, giúp tránh tạo dữ liệu trùng lặp, ví dụ như tạo đơn hàng hoặc thanh toán. Trace ID là một ID dùng để theo dõi luồng request xuyên suốt nhiều service trong hệ thống distributed. Nó không ngăn việc thực thi lặp lại mà chỉ giúp logging và monitoring dễ dàng hơn bằng cách liên kết tất cả log và request liên quan đến cùng một transaction.

[EN] An Idempotency Key is a unique key sent by the client with a request to ensure that an operation is executed only once, even if the request is retried due to network issues or timeouts. The server stores the key and the result; if the same key appears again, the server returns the previous result instead of processing it again, preventing duplicate actions such as creating an order or charging a payment. A Trace ID is an identifier used to track a request across multiple services in a distributed system. It does not prevent duplicate execution but helps with logging and monitoring by linking all logs and requests related to the same transaction.

[VN] Observer pattern hoạt động thế nào trong event-driven system?

[EN] How does the Observer pattern work in an event-driven system?

[VN] Observer pattern là một design pattern trong đó một đối tượng chính (subject hoặc publisher) giữ danh sách các đối tượng khác (observers hoặc subscribers) và tự động thông báo cho họ khi có sự kiện hoặc dữ liệu thay đổi. Trong hệ thống event-driven, khi một event xảy ra, publisher sẽ phát sự kiện và tất cả các subscriber đã đăng ký sẽ nhận được

thông báo và xử lý logic của mình. Cách này giúp giảm sự phụ thuộc trực tiếp giữa các thành phần, làm hệ thống dễ mở rộng và thêm listener mới mà không cần sửa code hiện có.

[EN] The Observer pattern is a design pattern where a main object (subject or publisher) keeps a list of other objects (observers or subscribers) and automatically notifies them when an event or data change happens. In an event-driven system, when an event occurs, the publisher emits the event and all subscribed observers receive the notification and execute their own logic. This approach reduces direct coupling between components and makes the system easier to extend because new listeners can be added without modifying existing code.

[VN] Unit of Work pattern giải quyết vấn đề gì trong transaction?

[EN] What problem does the Unit of Work pattern solve in transactions?

[VN] Unit of Work pattern giải quyết vấn đề quản lý transaction và nhiều thao tác database trong cùng một đơn vị công việc. Thay vì mỗi repository tự commit riêng lẻ, Unit of Work theo dõi tất cả các thay đổi (insert, update, delete) trong một transaction và chỉ commit một lần khi toàn bộ công việc hoàn thành. Nếu có lỗi xảy ra, hệ thống có thể rollback toàn bộ thay đổi để giữ dữ liệu nhất quán. Cách này giúp đảm bảo tính atomicity, giảm lỗi dữ liệu không đồng bộ và giúp quản lý transaction rõ ràng hơn trong các hệ thống lớn.

[EN] The Unit of Work pattern solves the problem of managing transactions and multiple database operations in a single unit of work. Instead of each repository committing changes separately, the Unit of Work tracks all changes (insert, update, delete) within one transaction and commits them only once when the whole operation is complete. If an error occurs, the system can roll back all changes to keep the data consistent. This approach ensures atomicity, reduces inconsistent data problems, and makes transaction management clearer in large systems.

[VN] CQRS pattern là gì và khi nào nên dùng?

[EN] What is the CQRS pattern and when should it be used?

[VN] CQRS (Command Query Responsibility Segregation) là pattern tách riêng phần ghi dữ liệu (command) và phần đọc dữ liệu (query) thành hai mô hình hoặc service khác nhau. Command xử lý các thay đổi dữ liệu như create, update, delete, còn query chỉ dùng để đọc dữ liệu. Việc tách này giúp tối ưu hệ thống vì logic đọc và ghi thường có yêu cầu khác nhau về performance và cấu trúc dữ liệu. CQRS nên dùng trong hệ thống lớn, có lượng đọc và ghi rất khác nhau, cần scale riêng cho read và write, hoặc khi cần mô hình dữ liệu đọc tối ưu hơn (ví dụ: dùng cache hoặc view riêng cho query).

[EN] CQRS (Command Query Responsibility Segregation) is a pattern that separates write operations (commands) and read operations (queries) into different models or services. Commands handle data changes such as create, update, and delete, while queries are only used to read data. This separation helps optimize the system because read and write operations often have different performance and data structure needs. CQRS is useful in large systems where read and write workloads are very different, when read and write need to scale independently, or when a specialized read model (such as cached views or optimized query structures) is required.

[VN] Saga pattern giúp quản lý distributed transaction như thế nào?

[EN] How does the Saga pattern manage distributed transactions?

[VN] Saga pattern là một cách quản lý distributed transaction trong hệ thống microservices bằng cách chia một transaction lớn thành nhiều transaction nhỏ cục bộ ở từng service. Mỗi service thực hiện một bước và nếu tất cả các bước thành công thì quy trình hoàn thành; nếu có bước nào thất bại thì hệ thống sẽ chạy compensation action (hành động hoàn tác) để rollback các bước trước đó. Saga thường được triển khai theo hai cách: choreography (các service giao tiếp qua event) hoặc orchestration (một service trung tâm điều phối). Cách này giúp hệ thống giữ được data consistency mà không cần distributed lock hay transaction toàn cục.

[EN] The Saga pattern manages distributed transactions in microservices systems by breaking one large transaction into multiple local transactions across different services. Each service completes one step, and if all steps succeed the process finishes; if any step fails, the system runs compensation actions to undo the previous steps. Saga is usually implemented in two ways: choreography (services communicate through events) or orchestration (a central service coordinates the workflow). This approach helps maintain data consistency without using distributed locks or global transactions.

[VN] Circuit Breaker pattern là gì và giúp hệ thống resilient ra sao?

[EN] What is the Circuit Breaker pattern and how does it make a system more resilient?

[VN] Circuit Breaker là một design pattern dùng để bảo vệ hệ thống khi gọi service bên ngoài bị lỗi hoặc chậm. Nó hoạt động giống một cầu dao điện: khi số lượng lỗi vượt ngưỡng, circuit sẽ chuyển sang trạng thái open và tạm thời chặn các request mới để tránh làm hệ thống quá tải. Sau một thời gian, circuit chuyển sang half-open để thử lại một vài request; nếu thành công thì quay lại closed. Pattern này giúp hệ thống resilient hơn vì tránh cascade failure, giảm tải khi dependency bị lỗi, và giúp service phục hồi nhanh hơn.

[EN] Circuit Breaker is a design pattern used to protect a system when calling external services that are slow or failing. It works like an electrical circuit breaker: when the number of failures passes a threshold, the circuit moves to the open state and temporarily blocks new requests to prevent overload. After some time, it moves to half-open to test a few requests, and if they succeed it returns to closed. This pattern makes the system more resilient by preventing cascade failures, reducing load when dependencies fail, and allowing services to recover faster.

[VN] Retry pattern nên dùng khi nào và có rủi ro gì?

[EN] When should the Retry pattern be used and what risks does it have?

[VN] Retry pattern được dùng khi một thao tác thất bại do lỗi tạm thời như network timeout, service quá tải tạm thời, hoặc lỗi kết nối. Hệ thống sẽ thử lại request sau một khoảng thời gian (thường dùng exponential backoff) để tăng khả năng thành công. Tuy nhiên, retry cũng có rủi ro: nếu retry quá nhiều có thể gây tăng tải hệ thống, tạo retry storm, hoặc gây duplicate operation nếu request không idempotent. Vì vậy cần giới hạn số lần retry, thêm delay hợp lý và thường kết hợp với circuit breaker.

[EN] The Retry pattern is used when an operation fails due to temporary errors such as network timeouts, temporary service overload, or connection issues. The system retries the request after some delay (often using exponential backoff) to increase the chance of success. However, retry also has risks: too many retries can increase system load, create a retry storm, or cause duplicate operations if the request is not idempotent. Therefore, it is important to limit retry attempts, add proper delays, and often combine it with a circuit breaker.

[VN] Bulkhead pattern giúp tránh cascading failure như thế nào?

[EN] How does the Bulkhead pattern prevent cascading failure?

[VN] Bulkhead pattern là một design pattern dùng để cô lập tài nguyên hoặc thành phần trong hệ thống để khi một phần bị lỗi thì không làm ảnh hưởng toàn bộ hệ thống. Ý tưởng giống như khoang kín trên tàu: nếu một khoang bị ngập nước, các khoang khác vẫn hoạt động. Trong hệ thống phần mềm, ta có thể tách thread pool, connection pool, hoặc service instance cho từng loại workload khác nhau. Nhờ vậy, nếu một service hoặc request type bị quá tải hoặc lỗi, nó chỉ ảnh hưởng phần tài nguyên của nó và không gây cascading failure cho toàn bộ hệ thống.

[EN] The Bulkhead pattern is a design pattern used to isolate resources or components in a system so that a failure in one part does not affect the whole system. The idea is similar to watertight compartments in a ship: if one compartment floods, the others can still operate. In software systems, we can separate thread pools, connection pools, or service instances

for different types of workloads. This way, if one service or request type becomes overloaded or fails, it only affects its own resources and prevents cascading failures across the entire system.

[VN] Event Sourcing pattern là gì?

[EN] What is the Event Sourcing pattern?

[VN] Event Sourcing là một pattern trong đó trạng thái của hệ thống được lưu bằng chuỗi các sự kiện (events) thay vì chỉ lưu trạng thái cuối cùng. Mỗi thay đổi trong hệ thống sẽ được ghi lại như một event bất biến (immutable event), và trạng thái hiện tại có thể được tái tạo bằng cách replay lại các event theo thứ tự thời gian. Cách này giúp dễ audit, debug và rebuild state khi cần, nhưng cũng làm hệ thống phức tạp hơn và cần cơ chế quản lý event và snapshot để đảm bảo hiệu năng.

[EN] Event Sourcing is a pattern where the system state is stored as a sequence of events instead of only storing the latest state. Every change in the system is recorded as an immutable event, and the current state can be rebuilt by replaying these events in order. This approach makes auditing, debugging, and rebuilding state easier, but it also increases system complexity and usually requires event management and snapshots to maintain good performance.

[VN] Outbox pattern giúp đảm bảo consistency giữa database và message queue như thế nào?

[EN] How does the Outbox pattern ensure consistency between the database and the message queue?

[VN] Outbox pattern giúp đảm bảo consistency giữa database và message queue bằng cách ghi message vào một bảng outbox trong cùng transaction với dữ liệu chính. Khi application cập nhật dữ liệu, nó cũng lưu event hoặc message vào bảng outbox trong cùng transaction, vì vậy hoặc cả hai cùng thành công hoặc cùng rollback. Sau đó, một background worker hoặc process riêng sẽ đọc các record trong bảng outbox và gửi chúng đến message queue (như Kafka hoặc RabbitMQ), rồi đánh dấu là đã gửi. Cách này tránh trường hợp data đã commit nhưng message không được gửi, giúp hệ thống distributed giữ được tính nhất quán.

[EN] The Outbox pattern ensures consistency between a database and a message queue by writing the message into an outbox table within the same transaction as the main data change. When the application updates data, it also saves an event or message to the outbox table in the same transaction, so both operations either succeed or rollback together. After that, a background worker or separate process reads records from the outbox table and publishes them to the message queue (such as Kafka or RabbitMQ), then marks them as

sent. This approach prevents cases where data is committed but the message is not published, helping maintain consistency in distributed systems.

[VN] Làm sao implement retry logic và exponential backoff?

[EN] How to implement retry logic and exponential backoff?

[VN] Để implement retry logic, bạn cần bọc đoạn code có thể fail (ví dụ gọi API hoặc query database) trong một vòng lặp thử lại khi gặp lỗi tạm thời, giới hạn số lần retry để tránh lặp vô hạn. Exponential backoff nghĩa là sau mỗi lần thất bại, thời gian chờ sẽ tăng theo cấp số nhân, ví dụ 1 giây, 2 giây, 4 giây, 8 giây, giúp giảm tải cho hệ thống và tránh spam request. Bạn cũng có thể thêm jitter (một khoảng thời gian ngẫu nhiên nhỏ) để tránh nhiều client retry cùng lúc. Trong Python, có thể tự viết bằng vòng lặp và `time.sleep` hoặc dùng thư viện như `tenacity` để quản lý retry linh hoạt hơn.

[EN] To implement retry logic, you wrap the code that may fail (such as an API call or database query) inside a loop that retries when a temporary error occurs, and limit the number of retries to avoid infinite loops. Exponential backoff means the waiting time increases exponentially after each failure, for example 1 second, 2 seconds, 4 seconds, 8 seconds, which reduces system load and prevents request spamming. You can also add jitter (a small random delay) to avoid many clients retrying at the same time. In Python, you can implement this manually using a loop with `time.sleep` or use a library like `tenacity` for more flexible retry management.

[VN] Circuit breaker pattern trong Python làm thế nào?

[EN] How to implement the circuit breaker pattern in Python?

[VN] Circuit breaker pattern trong Python được dùng để bảo vệ hệ thống khi một service bên ngoài liên tục lỗi. Bạn có thể triển khai bằng cách theo dõi số lần thất bại liên tiếp; nếu vượt quá ngưỡng, circuit sẽ chuyển sang trạng thái “open” và chặn các request mới trong một khoảng thời gian. Sau thời gian đó, chuyển sang trạng thái “half-open” để thử một vài request; nếu thành công thì đóng lại (“closed”), nếu tiếp tục lỗi thì mở lại. Có thể tự viết logic này bằng class quản lý trạng thái và thời gian, hoặc dùng thư viện như `pybreaker` để đơn giản hóa việc triển khai.

[EN] The circuit breaker pattern in Python is used to protect the system when an external service keeps failing. You can implement it by tracking consecutive failures; if the number exceeds a threshold, the circuit moves to the “open” state and blocks new requests for a period of time. After that timeout, it switches to “half-open” to test a few requests; if they succeed, it goes back to “closed”, otherwise it returns to “open”. You can implement this logic manually using a class to manage state and timing, or use a library like `pybreaker` to simplify the implementation.

[VN] CAP theorem là gì?

[EN] What is the CAP theorem?

[VN] CAP theorem là một nguyên lý trong hệ thống phân tán, nói rằng một hệ thống chỉ có thể đảm bảo tối đa hai trong ba yếu tố: Consistency (tính nhất quán), Availability (tính sẵn sàng) và Partition Tolerance (khả năng chịu phân vùng mạng). Consistency nghĩa là mọi node trả về cùng một dữ liệu tại cùng thời điểm. Availability nghĩa là hệ thống luôn phản hồi request, dù có lỗi xảy ra. Partition Tolerance nghĩa là hệ thống vẫn hoạt động dù có sự cố mạng giữa các node. Khi xảy ra network partition, hệ thống phải chọn giữa Consistency hoặc Availability. Vì vậy, trong thực tế, các hệ thống phân tán thường thiết kế theo CP hoặc AP tùy theo yêu cầu business.

[EN] The CAP theorem is a principle in distributed systems stating that a system can guarantee at most two out of three properties: Consistency, Availability, and Partition Tolerance. Consistency means all nodes return the same data at the same time. Availability means the system always responds to requests, even when failures occur. Partition Tolerance means the system continues to operate despite network failures between nodes. When a network partition happens, the system must choose between Consistency and Availability. Therefore, real-world distributed systems are usually designed as CP or AP depending on business requirements.

[VN] Eventual consistency là gì?

[EN] What is eventual consistency?

[VN] Eventual consistency là mô hình nhất quán trong hệ thống phân tán, trong đó dữ liệu giữa các node có thể tạm thời khác nhau, nhưng nếu không có cập nhật mới, tất cả các node sẽ dần dần đồng bộ và trở nên nhất quán theo thời gian. Điều này thường xảy ra trong các hệ thống AP theo CAP theorem, nơi ưu tiên availability hơn consistency ngay lập tức. Eventual consistency giúp hệ thống có hiệu năng cao và chịu lỗi tốt hơn, nhưng application cần chấp nhận việc đọc dữ liệu có thể chưa phải là phiên bản mới nhất trong một khoảng thời gian ngắn.

[EN] Eventual consistency is a consistency model in distributed systems where data across nodes may be temporarily different, but if no new updates occur, all nodes will eventually become consistent over time. This often happens in AP systems under the CAP theorem, where availability is prioritized over immediate consistency. Eventual consistency improves performance and fault tolerance, but the application must accept that reads may return slightly outdated data for a short period of time.

[VN] Message ordering trong Kafka đảm bảo như thế nào?

[EN] How is message ordering ensured in Kafka?

[VN] Trong Apache Kafka, message ordering chỉ được đảm bảo trong cùng một partition, không đảm bảo trên toàn bộ topic. Mỗi partition là một log tuần tự và Kafka ghi message theo thứ tự nhận được, vì vậy consumer đọc trong cùng partition sẽ nhận đúng thứ tự. Để đảm bảo ordering cho một key cụ thể (ví dụ user_id hoặc order_id), producer nên gửi message với cùng một key để Kafka hash và đưa vào cùng một partition. Nếu message được gửi vào nhiều partition khác nhau, thứ tự tổng thể sẽ không được đảm bảo. Vì vậy, thiết kế partition key rất quan trọng để giữ đúng message ordering theo yêu cầu business.

[EN] In Apache Kafka, message ordering is guaranteed only within a single partition, not across the entire topic. Each partition is a sequential log, and Kafka writes messages in the order they are received, so a consumer reading from the same partition will get messages in order. To guarantee ordering for a specific key (for example user_id or order_id), the producer should send messages with the same key so Kafka hashes them into the same partition. If messages are sent to different partitions, global ordering is not guaranteed. Therefore, choosing the correct partition key is important to maintain message ordering based on business requirements.

[VN] Exactly-once semantics có thật sự tồn tại không?

[EN] Does exactly-once semantics really exist?

[VN] Exactly-once semantics trong thực tế rất khó đạt được ở mức toàn bộ hệ thống phân tán, vì mạng có thể lỗi, message có thể bị gửi lại hoặc xử lý lại. Tuy nhiên, một số hệ thống như Apache Kafka hỗ trợ exactly-once semantics trong phạm vi nhất định, ví dụ đảm bảo message được ghi và xử lý đúng một lần trong Kafka khi cấu hình idempotent producer và transaction. Dù vậy, ở mức end-to-end (từ producer, message broker đến database), thường không thể đảm bảo tuyệt đối “exactly once”, mà thay vào đó người ta thiết kế hệ thống theo hướng at-least-once và kết hợp idempotency để đảm bảo kết quả cuối cùng chỉ được xử lý một lần về mặt business.

[EN] Exactly-once semantics is very difficult to achieve across an entire distributed system because networks can fail and messages can be retried or processed again. Some systems like Apache Kafka support exactly-once semantics within a limited scope, for example ensuring a message is written and processed only once inside Kafka when using idempotent producers and transactions. However, at the end-to-end level (from producer to broker to database), true “exactly once” is rarely guaranteed. Instead, systems are usually designed with at-least-once delivery and use idempotency to ensure the final business result happens only once.

[VN] Backpressure là gì?

[EN] What is backpressure?

[VN] Backpressure là cơ chế kiểm soát luồng dữ liệu khi một thành phần trong hệ thống xử lý chậm hơn tốc độ dữ liệu được gửi đến. Khi consumer hoặc service downstream không xử lý kịp, hệ thống sẽ gửi tín hiệu hoặc áp dụng cơ chế giới hạn để làm chậm producer, tránh việc quá tải, tràn bộ nhớ hoặc crash. Backpressure thường xuất hiện trong hệ thống streaming, message queue hoặc reactive system. Nếu không có backpressure, dữ liệu có thể bị dồn lại, gây tăng latency và làm hệ thống mất ổn định.

[EN] Backpressure is a flow control mechanism used when one component in a system processes data slower than the rate at which data is produced. When a consumer or downstream service cannot keep up, the system applies limits or sends signals to slow down the producer, preventing overload, memory overflow, or crashes. Backpressure is common in streaming systems, message queues, or reactive systems. Without backpressure, data can accumulate, increase latency, and make the system unstable.

[VN] Leader election trong distributed system

[EN] Leader election in distributed systems

[VN] Leader election là cơ chế trong distributed system dùng để chọn ra một node làm leader để điều phối hoặc xử lý một số tác vụ quan trọng, trong khi các node khác đóng vai trò follower. Leader thường chịu trách nhiệm ghi dữ liệu, điều phối job hoặc quản lý state chung để tránh xung đột. Khi leader bị lỗi hoặc mất kết nối, hệ thống sẽ tự động bầu chọn một leader mới thông qua các thuật toán như Raft hoặc Paxos. Cơ chế này giúp đảm bảo tính nhất quán và tránh split-brain, nhưng cần thiết kế cẩn thận để xử lý network partition và đảm bảo high availability.

[EN] Leader election is a mechanism in distributed systems used to select one node as the leader to coordinate or handle critical tasks, while other nodes act as followers. The leader is usually responsible for writing data, coordinating jobs, or managing shared state to avoid conflicts. If the leader fails or loses connection, the system automatically elects a new leader using algorithms such as Raft or Paxos. This mechanism ensures consistency and prevents split-brain situations, but it must be carefully designed to handle network partitions and maintain high availability.

[VN] Data sharding là gì?

[EN] What is data sharding?

[VN] Data sharding là kỹ thuật chia dữ liệu lớn thành nhiều phần nhỏ (shard) và lưu trên nhiều database hoặc server khác nhau để tăng khả năng scale và hiệu năng. Mỗi shard chứa một phần dữ liệu dựa trên shard key, ví dụ user_id hoặc region. Khi có request, hệ thống sẽ dựa vào shard key để định tuyến đến đúng shard thay vì truy vấn toàn bộ dữ liệu. Sharding

giúp phân tán tải, tăng throughput và tránh giới hạn của một database đơn lẻ. Tuy nhiên, nó làm hệ thống phức tạp hơn, đặc biệt khi cần join dữ liệu giữa các shard hoặc thay đổi shard key.

[EN] Data sharding is a technique that splits large datasets into smaller pieces (shards) and stores them across multiple databases or servers to improve scalability and performance. Each shard contains a subset of data based on a shard key, such as user_id or region. When a request comes in, the system uses the shard key to route the query to the correct shard instead of scanning all data. Sharding distributes load, increases throughput, and avoids the limits of a single database. However, it makes the system more complex, especially when joining data across shards or changing the shard key.

[VN] Clock skew ảnh hưởng thế nào?

[EN] How does clock skew affect systems?

[VN] Clock skew là hiện tượng thời gian giữa các server trong hệ thống phân tán không đồng bộ chính xác với nhau. Điều này có thể gây lỗi trong việc sắp xếp event, so sánh timestamp, hoặc xử lý timeout và token hết hạn. Ví dụ, nếu một server có thời gian nhanh hơn, nó có thể tạo log hoặc event với timestamp “tương lai”, làm sai thứ tự xử lý. Clock skew cũng ảnh hưởng đến cơ chế consensus, transaction và cache expiration. Để giảm vấn đề này, hệ thống thường dùng NTP để đồng bộ thời gian và hạn chế phụ thuộc hoàn toàn vào timestamp khi thiết kế logic nghiệp vụ.

[EN] Clock skew is the situation where system clocks on different servers in a distributed system are not perfectly synchronized. This can cause problems when ordering events, comparing timestamps, or handling timeouts and token expiration. For example, if one server's clock is ahead, it may generate logs or events with a “future” timestamp, breaking the correct processing order. Clock skew can also affect consensus mechanisms, transactions, and cache expiration. To reduce this issue, systems often use NTP to synchronize clocks and avoid relying fully on timestamps in business logic design.

[VN] IBM MQ vs Kafka

[EN] IBM MQ vs Kafka

[VN] IBM MQ là hệ thống message queue truyền thống, tập trung vào việc gửi và nhận message một cách tin cậy giữa các hệ thống, thường dùng trong môi trường enterprise với yêu cầu đảm bảo giao dịch và thứ tự message. Message thường được tiêu thụ rồi xóa khỏi queue. Trong khi đó, Apache Kafka là nền tảng streaming phân tán, được thiết kế để xử lý lượng dữ liệu lớn theo thời gian thực. Kafka lưu message trong một khoảng thời gian nhất định và cho phép nhiều consumer đọc lại dữ liệu. Tóm lại, IBM MQ phù hợp cho tích hợp

hệ thống và giao dịch truyền thống, còn Kafka phù hợp cho xử lý dữ liệu lớn, streaming và kiến trúc event-driven.

[EN] IBM MQ is a traditional message queue system focused on reliable message delivery between systems, often used in enterprise environments that require transaction support and message ordering. Messages are usually consumed and then removed from the queue. In contrast, Apache Kafka is a distributed streaming platform designed to handle large volumes of real-time data. Kafka stores messages for a configurable period and allows multiple consumers to read the same data. In short, IBM MQ is suitable for traditional system integration and transactions, while Kafka is better for big data processing, streaming, and event-driven architecture.

[VN] Luồng giao dịch kết hợp giữa Core Banking, IBM MQ và Kafka

[EN] Transaction flow combining Core Banking, IBM MQ, and Kafka

[VN] Khi bạn thực hiện lệnh chuyển tiền trên ứng dụng di động, yêu cầu sẽ được hệ thống trung gian gửi vào hàng đợi của IBM MQ. Tại đây, IBM MQ đóng vai trò quan trọng trong việc đảm bảo dữ liệu giao dịch luôn an toàn, không bị thất lạc và chỉ được gửi duy nhất một lần. Sau đó, hệ thống Core Banking sẽ tiếp nhận yêu cầu từ hàng đợi này để kiểm tra số dư, thực hiện hạch toán ghi nợ, ghi có và cập nhật cơ sở dữ liệu, đồng thời gửi phản hồi xác nhận qua một hàng đợi khác để thông báo cho người dùng. Ngay khi giao dịch hoàn tất thành công, công cụ Kafka Connect sẽ tự động sao chép thông tin giao dịch từ IBM MQ và đẩy vào Kafka mà không làm ảnh hưởng đến hiệu suất của hệ thống lõi. Cuối cùng, Kafka đóng vai trò là nền tảng điều phối dữ liệu để các ứng dụng khác như hệ thống chống rửa tiền (AML) thực hiện phát hiện gian lận, gửi thông báo biến động số dư tức thì qua tin nhắn hoặc cập nhật báo cáo tài chính thời gian thực. Tóm lại, IBM MQ đảm nhận việc xử lý giao dịch an toàn và chính xác, trong khi Kafka chịu trách nhiệm lan tỏa và phân tích dữ liệu nhanh chóng cho toàn bộ hệ thống.

[EN] When you make a transfer on a mobile app, the request is sent by middleware to an IBM MQ queue. In this step, IBM MQ plays a vital role by ensuring that transaction data is always safe, never lost, and delivered exactly once. Next, the Core Banking system retrieves the request from the queue to check the balance, process the debit and credit entries, and update the database, while also sending a confirmation back through a response queue to notify the user. As soon as the transaction is successful, Kafka Connect automatically copies the transaction info from IBM MQ and pushes it into Kafka without slowing down the core system. Finally, Kafka acts as a data streaming platform where other applications can subscribe to perform real-time tasks like fraud detection (AML), sending instant balance alerts via SMS or apps, and updating financial reports

immediately. In short, IBM MQ focuses on secure and accurate transaction processing, while Kafka handles fast data distribution and analytics for the entire system.

V. Distributed Systems & Transactions

Làm sao để xử lý distributed transaction? So sánh 2PC vs Saga pattern.

Circuit Breaker là gì? Khi nào cần?

Idempotency trong API là gì? Áp dụng vào payment/booking như thế nào?

Nếu một service downstream bị chậm, hệ thống sẽ bị ảnh hưởng ra sao? Cách mitigate?

Idempotency Key vs Trace ID

Observer pattern hoạt động thế nào trong event-driven system?

Unit of Work pattern giải quyết vấn đề gì trong transaction?

CQRS pattern là gì và khi nào nên dùng?

Saga pattern giúp quản lý distributed transaction như thế nào?

Circuit Breaker pattern là gì và giúp hệ thống resilient ra sao?

Retry pattern nên dùng khi nào và có rủi ro gì?

Bulkhead pattern giúp tránh cascading failure như thế nào?

Event Sourcing pattern là gì?

Outbox pattern giúp đảm bảo consistency giữa database và message queue như thế nào?

Làm sao implement retry logic và exponential backoff?

Circuit breaker pattern trong Python làm thế nào?

CAP theorem là gì?

Eventual consistency là gì?

Message ordering trong Kafka đảm bảo như thế nào?

Exactly-once semantics có thật sự tồn tại không?

Backpressure là gì?

Leader election trong distributed system.

Data sharding là gì?

Clock skew ảnh hưởng thế nào?

IBM MQ vs Kafka

Luồng giao dịch kết hợp giữa Core Banking, IBM MQ và Kafka